Programowanie obiektowe

Wykład 6

Dziedziczenie II – Polimorfizm

Polimorfizm

- **Polimorfizm** – jedna z podstawowych cech obiektowego paradygmatu programowania, obok abstrakcji, dziedziczenia i enkapsulacji (hermetyzacji).
- Dosłownie oznacza „wiele form" i odnosi się do możliwości używania jednego interfejsu (np. metody) do reprezentowania różnych zachowań, zależnie od kontekstu.

Rodzaje polimorfizmu:

1. **Polimorfizm statyczny (*compile-time*)** – przeważnie odnosi się do przeciążania metod (ang. ***method overloading***) lub operatorów. Przykład:

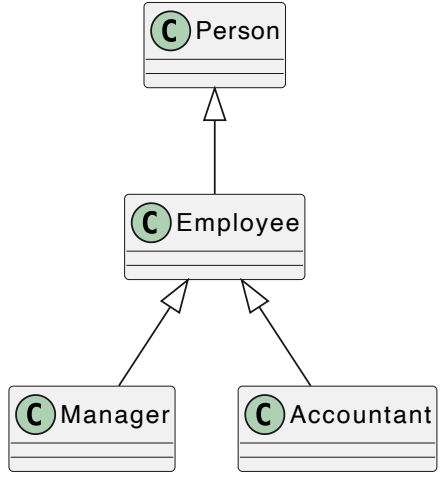
```
int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}
```

2. **Polimorfizm dynamiczny (*run-time*)** – osiągany dzięki dziedziczeniu i nadpisywaniu metod (ang. ***method overriding***).

Dziedziczenie

Mechanizm pozwalający tworzyć **hierarchie klas** oraz modelować **relacje** między typami.

- Klasy mogą tworzyć strukturę hierarchiczną (np. `Person`, `Employee`, `Manager`)
- Klasa pochodna jest bardziej szczegółowym przypadkiem klasy bazowej (**specjalizacja**)
- Klasa bazowa może reprezentować wspólne cechy wielu klas (**generalizacja**)
- Dziedziczenie powinno odzwierciedlać relację „X jest rodzajem Y" (**is-a**)
  - `Manager` jest rodzajem `Employee`
  - `Employee` jest rodzajem `Person`



Dziedziczenie typu

- Klasa dziedziczy nie tylko składowe ale również typ.

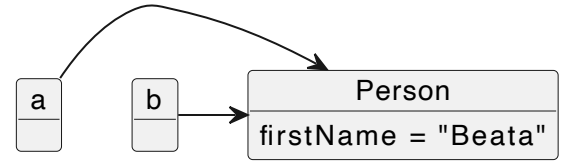
```
class MailService {
    public static void SendMessage(Employee employee, string message) {
        // Logika wysyłania wiadomości do pracownika
    }
}

Manager manager = new Manager("Jan", "Kowalski", "janek@company.com");
// Dozwolone, ponieważ menadżer jest pracownikiem
MailService.SendMessage(manager, "Wesołych świąt dla załogi!");
```

- Obiekt klasy pochodnej może być traktowany jako obiekt klasy bazowej.
  - Metoda `SendMessage` oczekiwała klasy bazowej (`Employee`), a dostarczyliśmy jej obiekt klasy pochodnej (`Manager`).
  - Klasa `Manager` jest traktowana jako typ `Employee`, czyli można jej używać wszędzie tam, gdzie oczekiwany jest `Employee`.

Przypisywanie referencji

- Gdy posługujemy się referencjami (oraz wskaźnikami), wiele referencji może wskazywać ten sam obiekt:

```
Person a = new Person();
a.firstName = "Anna";

Person b = a;
b.firstName = "Beata";
```



- `a` to referencja do obiektu `Person`.
- `b = a` oznacza, że `b` wskazuje dokładnie ten sam obiekt co `a`.
- `b = a` nie tworzy nowego obiektu, a jedynie nową referencję do tego samego obiektu.

Klasa, która ma wiele typów, może być wskazywana referencję dowolnego typu, który posiada

```
Manager manager = new Manager("Jan", "Kowalski", "janek@company.com");
Employee employee = manager;
Person person = manager;
```

Możemy przypisać **obiekt klasy pochodnej** do **referencji klasy bazowej**.

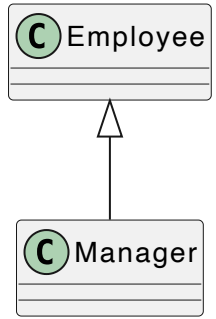
Rzutowanie w górę (*upcasting*)

Z typu bardziej szczegółowego → do typu bardziej ogólnego

- Przypisanie referencji obiektu klasy pochodnej do zmiennej typu klasy bazowej.

```
Employee e = new Manager("Jan", "Kowalski", "janek@company.com");
```

- Typ zmiennej określa, co możemy zrobić z obiektem, a nie czym ten obiekt naprawdę jest
  - zmienna `e` ma typ `Employee`
  - ale obiekt nie zmienia swojego rzeczywistego typu (to nadal `Manager`)
- To tylko zmiana **sposobu patrzenia** na obiekt – tak jak by był obiektem klasy bazowej
- Możemy używać tylko tych metod, które są dostępne w typie `Employee`



- Rzutowanie w górę *zawsze jest bezpieczne* – każdy obiekt klasy pochodnej jest też obiektem klasy bazowej.
- Nie wymaga jawnego rzutowania – kompilator wykonuje to automatycznie.

Zastosowania rzutowania w górę

- Przekazywanie różnych klas pochodnych jako argumenty do metod oczekujących typu bazowego.

```
class MailService {
    public static void SendMessage(Employee employee, string message) {
        // Ta metoda pozwala na wysłanie wiadomości do dowolnego pracownika
    }
}
```

- Przechowywanie obiektów różnych typów pochodnych w jednej kolekcji typów ogólnych.

```
Employee[] employees = new Employee[4];

employees[0] = new Developer("Alice", "alice@company.com");
employees[1] = new Accountant("Bob", "bob@company.com");
employees[2] = new Developer("Charlie", "charlie@company.com");
employees[3] = new Accountant("Dana", "dana@company.com");

foreach (Employee employee in employees)
{
    Console.WriteLine($"{employee.FirstName} ({employee.Email})");
    employee.DoWork();
}
```

Polimorfizm

- Obiekty różnych klas możemy traktować jako jeden typ bazowy:

```
Employee e1 = new Employee(...);
Employee e2 = new Manager(...);
Employee e3 = new Developer(...);
```

- Pozwala nam to pisać kod operujący na wspólnym typie:

```
e1.getSalary();
e2.getSalary();
e3.getSalary();
```

- Na poziomie typu zmiennej (`Employee`) nie wiemy, jakiego dokładnie typu jest obiekt.
- Chcemy wykonywać odpowiednią operację, w zależności od rzeczywistego typu obiektu.

Polimorfizm – zdolność traktowania różnych obiektów w jednolity sposób, przy jednoczesnym zachowaniu ich **specyficznego zachowania**.

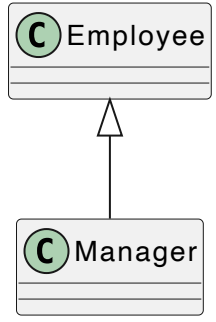
Rzutowanie w dół (downcasting)

Z typu bardziej ogólnego → do typu bardziej szczegółowego

- Przypisanie referencji typu klasy bazowej do zmiennej typu klasy pochodnej.

```
Employee e = new Manager("Jan", "Kowalski", "janek@company.com");
Manager m = (Manager)e;
```

- Jest to próba odzyskania konkretnego typu:
  - próbujemy odzyskać bardziej szczegółowy typ, utracony wskutek rzutowania w górę
  - zakładamy, że obiekt jest konkretnego typu
- To również wyłącznie zmiana **sposobu patrzenia** na obiekt – sam obiekt nie ulega przekształceniu



- Rzutowanie w górę *Może być niebezpieczne* – poprawne tylko wtedy, gdy rzeczywisty obiekt jest instancją klasy pochodnej.
- Wymaga **jawnego rzutowania** (`(TypPochodny)referencjaBazowa`).
- Może prowadzić do błędu wykonania (`ClassCastException` w Java lub `InvalidCastException` w C#), jeśli typ obiektu nie pasuje.

Zastosowanie rzutowania w dół

- Odzyskanie dostępu do metod i pól specyficznych dla klasy pochodnej lub implementacja zachowania zależnego od konkretnego typu obiektu.

```
Employee[] employees = new Employee[4];

...

foreach (Employee employee in employees)
{
    Accountant accountant = (Accountant)employee;
    accountant.GenerateReport();
}
```

- Ale skąd mamy wiedzieć czy dany pracownik faktycznie jest księgowym?

```
Employee e1 = new Accountant("Bob");   // ✅ Upcasting (bezpieczny, automatyczny)
((Accountant)e1).GenerateReport();     // ✅ Downcasting (działa, bo Bob to księgowy)

Employee e2 = new Developer("Alice");  // ✅ Upcasting (bezpieczny, automatyczny)
((Accountant)e2).GenerateReport();     // ❌ Downcasting niepoprawny
```

Informacja statyczna

- **Informacja statyczna** (typ statyczny), to typ, który zna kompilator – na podstawie deklaracji zmiennej w kodzie.
  - Jest znana **w czasie kompilacji**
  - Nie musi odpowiadać rzeczywistemu typowi obiektu podczas wykonania
  - Służy do sprawdzania poprawności składni (np. czy metoda istnieje) i kontroli typów przy rzutowaniu.

Przykład:

```
Employee e = new Accountant("Bob");
```

- Statyczny typ `e` to `Employee` → kompilator wie tylko, że `e` ma metody klasy `Employee`.

Informacja dynamiczna

- **Informacja dynamiczna** (typ dynamiczny) to rzeczywisty typ obiektu w pamięci – czyli klasa, z której obiekt został utworzony w czasie działania programu.
  - Jest znana **w czasie wykonania (runtime)**
  - Wykorzystywana np. przez dynamiczne wiązanie metod (polimorfizm), `instanceof`, `getClass()` itd.

Przykład:

```
Employee e = new Accountant("Bob");
Accountant a = (Accountant)e;
a.GenerateReport();
```

- Dynamiczny typ `e` to `Accountant` (dlatego rzutowanie było możliwe).

RTTI

- **RTTI** (*Run-Time Type Information*) – informacja o typie w czasie wykonywania.
- To mechanizm, który pozwala programowi **poznać rzeczywisty (dynamiczny) typ obiektu podczas działania**, niezależnie od jego typu statycznego.

RTTI pozwala:

- określić typ dynamiczny obiektu,
- bezpiecznie wykonywać rzutowanie w dół (*downcasting*),
- obsługiwać polimorfizm.

Jak to działa?

- W większości języków OOP RTTI opiera się na **dodatkowej strukturze danych** (np. tablica vtable, wskaźnik na typ runtime), która jest dołączana do każdego obiektu posiadającego metody wirtualne.

W uproszczeniu:

- Obiekt wie, z jakiej klasy pochodzi.
- System może sprawdzić ten typ, mimo że referencja może być ogólniejsza.

RTTI w C++

- RTTI jest **domyślnie włączone**, ale można je **wyłączyć** np. przy kompilacji z flagą `-fno-rtti` (np. w GCC).
- Wymaga użycia **co najmniej jednej metody wirtualnej** w klasie bazowej.
- Główne narzędzia RTTI w C++:
  - `typeid` – zwraca obiekt typu `std::type_info` reprezentujący typ dynamiczny.
  - `dynamic_cast` – umożliwia bezpieczne rzutowanie w dół.

```cpp
#include <iostream>
#include <typeinfo>

class Employee {
public:
    // wirtualny destruktor
    virtual ~Employee() {}
};

class Accountant : public Employee {
public:
    void generateReport() {
        // Logika generowania raportu
    }
};
```

```cpp
// Client:
Employee* e = new Accountant("Bob");
Accountant *a;

// RTTI - sprawdzenie typu
if (typeid(*e) == typeid(Accountant)) {
    // to jest ksiegowy

    // RTTI - bezpieczne rzutowanie
    a = dynamic_cast<Accountant*>(e);
    if (a != nullptr) {
        // bezpieczne wywołanie
        a->generateReport();
    }
}
delete e;
```

RTTI w Javie

- RTTI w Javie jest **wbudowane i zawsze dostępne**.
- Główne narzędzia RTTI w Java:
  - `instanceof` – sprawdza, czy obiekt jest instancją klasy lub jej podklasy.
  - `getClass()` – zwraca `Class<?>`, który reprezentuje typ obiektu.

```java
class Employee {

}

class Accountant extends Employee {
    public void generateReport() {
        // Logika generowania raportu
    }
}
```

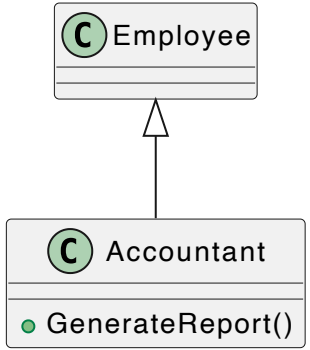
```java
// Client:
Employee e = new Accountant("Bob");
Accountant a;

// RTTI - sprawdzenie typu
if (e instanceof Accountant) {
    // to jest ksiegowy

    // bezpieczne rzutowanie
    a = (Accountant)e;

    // bezpieczne wywołanie
    a.generateReport();
}
```

- `e` ma **statyczny typ** `Employee`, ale **dynamicznie** `Accountant`.
- Dzięki RTTI (`instanceof`) możemy sprawdzić jego typ w czasie działania.

RTTI w C#

- W C# RTTI również jest **wbudowane i domyślnie aktywne**.
- Główne narzędzia RTTI w C#:
  - `is` – sprawdza typ.
  - `as` – bezpieczne rzutowanie (zwraca `null` zamiast wyjątku).
  - `GetType()` – zwraca `System.Type` obiektu.

```
Employee e = new Accountant("Bob");

// RTTI - sprawdzenie typu
if (e is Accountant) {
    // to jest ksiegowy
}

// RTTI - bezpieczne rzutowanie
Accountant a = e as Accountant;

if(a != null) {
    // bezpieczne wywołanie
    a.GenerateReport();
}
```



Zastosowania RTTI – przykład

```
Random rand = new Random();
Employee[] employees = new Employee[10];

// Wypełnij tablicę losowymi pracownikami
for (int i = 0; i < employees.Length; i++)
{
    if (rand.Next(2) == 0)
    {
        employees[i] = new Developer($"Dev{i}", $"dev{i}@company.com");
    }
    else
    {
        employees[i] = new Accountant($"Acc{i}", $"acc{i}@company.com");
    }
}

// Wywołaj GenerateReport tylko dla księgowych
foreach (Employee employee in employees)
{
    if (employee is Accountant)
    {
        Accountant accountant = employee as Accountant;
        accountant.GenerateReport();
    }
}
```

Stosowanie RTTI

- Kiedy używać?
  - Gdy chcemy *bezpiecznie* sprawdzić typ dynamiczny przed rzutowaniem w dół.
  - Gdy różnicujemy swoje zachowanie zależnie od faktycznego typu obiektu na którym operujemy

```
class MailService {
    public static void sendMessage(Employee employee, String message) {
        if(employee instanceof Developer) {
            // specjalny nagłówek dla programistów
            System.out.println(" ... ");
        }
        // część wiadomości wysyłana do wszystkich
        System.out.println(" ... ");
    }
}
```

- Dlaczego warto **unikać RTTI**?
  - Bywa kosztowne w czasie działania.
  - Może to wskazywać na **zły projekt** (np. brak polimorfizmu tam, gdzie powinien być).
- **Metody wirtualne** oferują znacznie przyjaźniejszy i przejrzystszy mechanizm.

Metody wirtualne

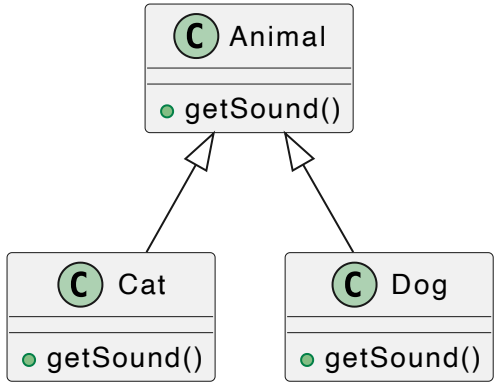
- **Metoda wirtualna** jest to metoda w klasie bazowej, którą można nadpisać w klasie pochodnej i która będzie wywoływana dynamicznie (w czasie działania), gdy korzystamy z referencji lub wskaźnika do klasy bazowej.

```
public class Animal {
    public String getSound() { return "Some generic animal sound."; }
}
public class Dog extends Animal {
    public String getSound() { return "Woof!"; }
}
public class Cat extends Animal {
    public String getSound() { return "Meow!"; }
}

Animal animal = new Dog();
// Która wersja metody zostanie wywołana?
System.out.println(animal.getSound());
```

- **Mechanizm wirtualny** umożliwia wywołania właściwej wersji metody (tej z klasy rzeczywistego obiektu), nawet jeśli używamy wskaźnika lub referencji typu bazowego.
- Mechanizm wirtualny pozwala, by wywołana została metoda `Dog.getSound()`, mimo że referencja jest typu `Animal`.

Wiązanie metod

- **Wiązanie metod** (ang. *method binding*) to proces **skojarzenia wywołania metody z jej konkretną implementacją** w kodzie.
- Gdy piszemy w kodzie `animal.getSound()`, to komputer musi rozstrzygnąć, **którą konkretnie wersję metody** (z której klasy) wywołać.



- Jeżeli zajmuje się tym kompilator, jest to **wiązanie wczesne** (ang. *early binding*) lub wiązanie statyczne
- Jeżeli odbywa się to podczas wykonania programu, jest to **wiązanie późne** (ang. *late binding*) lub wiązanie dynamiczne

Wiązanie wczesne (*early binding / static binding*)

- To wiązanie, które odbywa się **w czasie kompilacji** – kompilator wie dokładnie, **która metoda zostanie wywołana**, bo typ jest znany statycznie.

Przykład:

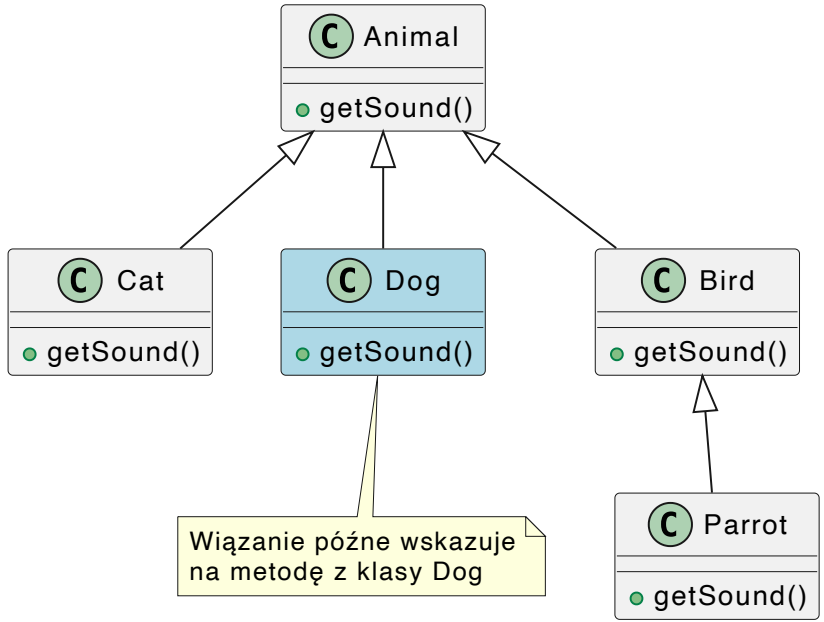
```
class Animal {
public:
    void move() {
        cout << "The animal moves." << endl;
    }
};

int main() {

    Animal *animalPtr = new Animal();
    animalPtr->move();
    delete animalPtr;

    return 0;
}
```

Wiązanie późne (late binding / dynamic binding)

- To wiązanie, które odbywa się **w czasie działania programu (*runtime*)**. Wybrana metoda zależy od **dynamicznego typu obiektu**, a nie tylko od statycznego typu referencji.

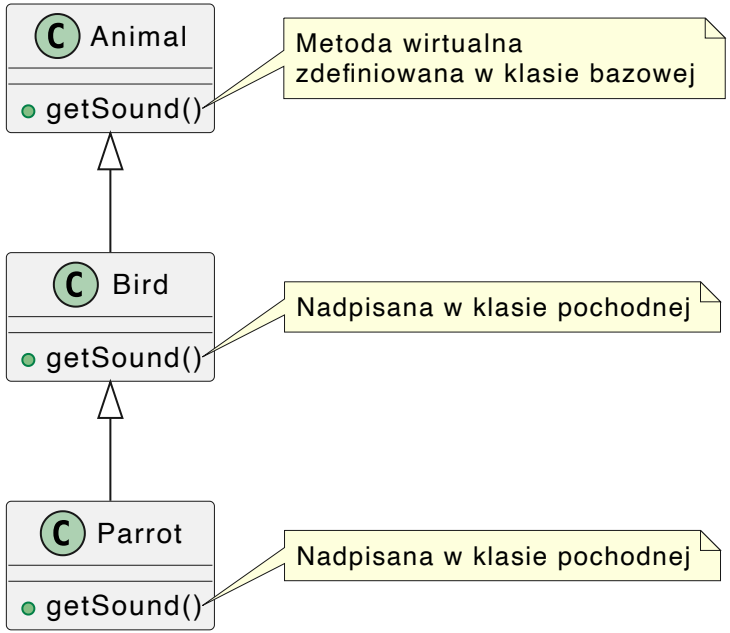
Przykład:

```
Animal animal = new Dog();
System.out.println(animal.getSound());
```



Łańcuch metod wirtualnych

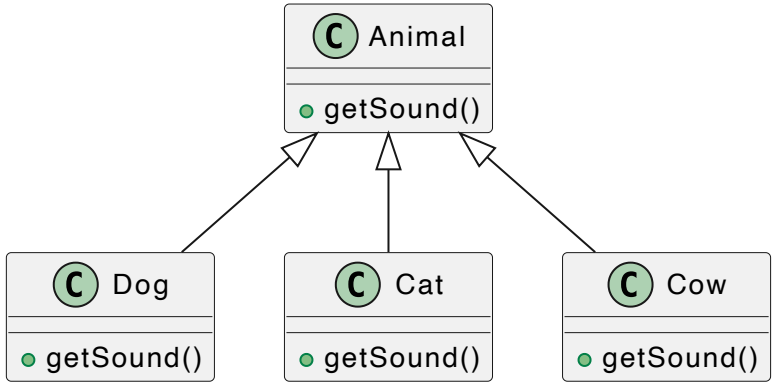
- Każda kolejna klasa pochodna może nadpisać metodę wirtualną z klasy bazowej – w ten sposób tworzy się **łańcuch nadpisywania**.
- Jest to łańcuch metod wirtualnych – ciąg kolejnych implementacji tej samej metody, które **nadpisują się w klasach potomnych** w hierarchii dziedziczenia.
- Wszystkie metody mają tę samę nazwę i sygnaturę.
- Łańcuch jest rozstrzyganych przez **dynamiczne wiązanie** w czasie wykonania.



Polimorfizm dynamiczny

- Dzięki dziedziczeniu i nadpisywaniu metod (ang. *method overriding*) osiągamy polimorfizm dynamiczny: obiektami różnych klas posługujemy się przez **wspólny interfejs**.

```
Animal[] animals = {
   new Dog(),
   new Cat(),
   new Cow()
};

for (Animal animal : animals) {
   System.out.println(animal.getSound());
}
```



- Obiekty różnych klas przechowujemy w jednej tablicy (rzutowanie w górę, naturalna konsekwencja dziedziczenia typu).
- Wywołujemy metodę z odpowiedniej klasy dzięki mechanizmowi wirtualnemu i korzystając z informacji dynamicznej.
  - Bez rzutowania w dół i jawnego sprawdzania typu.

Metody wirtualne w C++

- Słowo kluczowe `virtual` w klasie bazowej sprawia, że metoda korzysta z **mechanizmu wirtualnego**.

```
class Animal {
    public:
    virtual string getSound() {
        return "Nieokreślony dźwięk";
    }
};
```

- Od C++11 można (i warto) dodać `override` w klasie pochodnej.

```
class Dog : public Animal {
public:
    string getSound() override {
        return "Hau hau!";
    }
};
```

- Jest to **nadpisywanie** metody (*overriding*) – dostarczamy nową implementację metody odziedziczonej.
- Jeśli metoda nie jest `virtual`, to będziemy mieć do czynienia z **ukrywaniem** metody bazowej.

Metody wirtualne w Javie

- Domyślnie **wszystkie metody są wirtualne** (poza `final`, `static` lub `private`).
  - Metody niewirtualne mogą być wydajniejsze.
- Zaleca się użycie adnotacji `@Override`, ale nie jest obowiązkowa.

```
class Animal {
    String getSound() {
        return "Nieokreślony dźwięk";
    }
}

class Dog extends Animal {
    @Override
    String getSound() {
        return "Hau hau!";
    }
}
```

- Metoda finalna (`final`) – nie można jej nadpisywać.
- Klasa finalna – nie można z niej dziedziczyć.

Metody wirtualne w C#

- Domyślnie metody wiązane są **statycznie**.
- Metody wchodzące w skład mechanizmu wirtualnego należy oznaczać jako `virtual` (w klasie bazowej) i `override` (w klasach pochodnych).

```
class Animal {
    public virtual string GetSound() {
        return "Nieokreślony dźwięk";
    }
}

class Dog : Animal {
    public override string GetSound() {
        return "Hau hau!";
    }
}
```

- Zamiast **przeciążać** metodę, możemy ją **ukryć** (nawet jeśli w klasie bazowej zadeklarowaliśmy ją jako wirtualną) – w tym celu używamy jawnie słowa kluczowego `new`:

```
public new string GetSound() { ... }
```

- Metoda `sealed` – nie można jej nadpisać. Klasa `sealed` – nie można z niej dziedziczyć.

Nadpisywanie vs ukrywanie w C#

```
class Person
{
    public virtual void Introduce()  { Console.WriteLine("I'm a person."); }
    public virtual void SayHello()   { Console.WriteLine("Hello from Person."); }
}
class Employee : Person
{
    // nadpisywanie (override)
    public override void Introduce() { Console.WriteLine("I'm an employee."); }
    // ukrywanie (new)
    public new void SayHello()       { Console.WriteLine("Hello from Employee."); }
}

// Client:
Person person = new Employee();

person.Introduce();  // "I'm an employee." (działa polimorfizm)
person.SayHello();   // "Hello from Person." (ukrywanie, zależy od typu referencji)

Console.WriteLine();

Employee employee = new Employee();

employee.Introduce(); // "I'm an employee."
employee.SayHello();  // "Hello from Employee." (pełny dostęp do ukrytej metody)
```

Wywoływanie metody z klasy bazowej

- Gdy chcemy rozszerzyć lub uzupełnić logikę z klasy bazowej, zamiast ją całkowicie zastąpić:
  - Zachowanie wspólne dla wszystkich klas jest implementowane w klasie bazowej.
  - Tworzymy nową wersję metody (*override*) w klasie pochodnej.
  - Ręcznie wywołujemy metodę z klasy bazowej (`super` / `base` / `Klasa::`).
  - Dzięki temu zachowujemy oryginalne zachowanie i dodajemy nowe.

```
public class FileWriter
{
    public virtual void Save(string content) {
        // Bazowa implementacja zapisu danych do pliku
    }
}
public class EncryptedFileWriter : FileWriter
{
    public override void Save(string content)
    {
        string encrypted = Encrypt(content);  // Szyfrujemy przed zapisem
        base.Save(encrypted);                 // Wywołujemy metodę z klasy bazowej
    }
    private static string Encrypt(string input) {
        // Implementacja szyfrowania
    }
}
```

`Object` w Javie i `object` w C#

- W obu językach `Object` (Java) i `object` (C#) to **najbardziej ogólna klasa bazowa**, z której **dziedziczą wszystkie inne klasy** – jawnie lub domyślnie.
- Pozwala nam to:
  - przypisać **dowolny obiekt** do zmiennej typu `Object` / `object`,
  - traktować każdy obiekt jako `Object` bez rzutowania,
  - pisać kod operujący na dowolnych obiektach.

Przykład w Javie:

```
class PrintStream {
    public void println(Object o) { ... }
}
```

Przykład w C#:

```
class Console {
    public void WriteLine(object o) { ... }
}
```

- To jest najbardziej ogólny typ – ale też najmniej informacyjny. Pozwala traktować wszystkie obiekty jednakowo (np. trzymać dowolne obiekty razem), ale kosztem utraty informacji o ich konkretnym typie.
- Klasa `Object` definiuje też kilka podstawowych metod, np. `toString()`, które możemy nadpisywać w klasach pochodnych.

Typy podstawowe jako obiekt w Javie i C#

- W **C#**, nawet typy wartości (np. `int`, `bool`) mogą być traktowane jako `object` dzięki **boxingowi**.

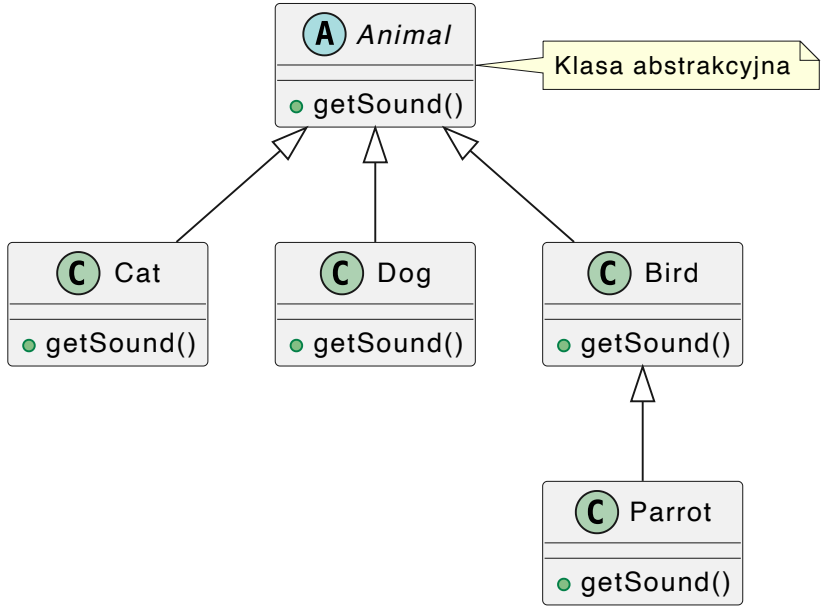
```
int num = 42;
object boxed = num;  // boxing: int → object

int unboxed = (int)boxed;  // unboxing: object → int
```

- W **Java**, wszystko poza typami prymitywnymi (`int`, `double`, itd.) dziedziczy po `Object`. Dla prymitywów są klasy opakowujące (np. `Integer`).

```
int num = 42;
Integer boxed = num;  // autoboxing

int unboxed = boxed;  // auto-unboxing
```

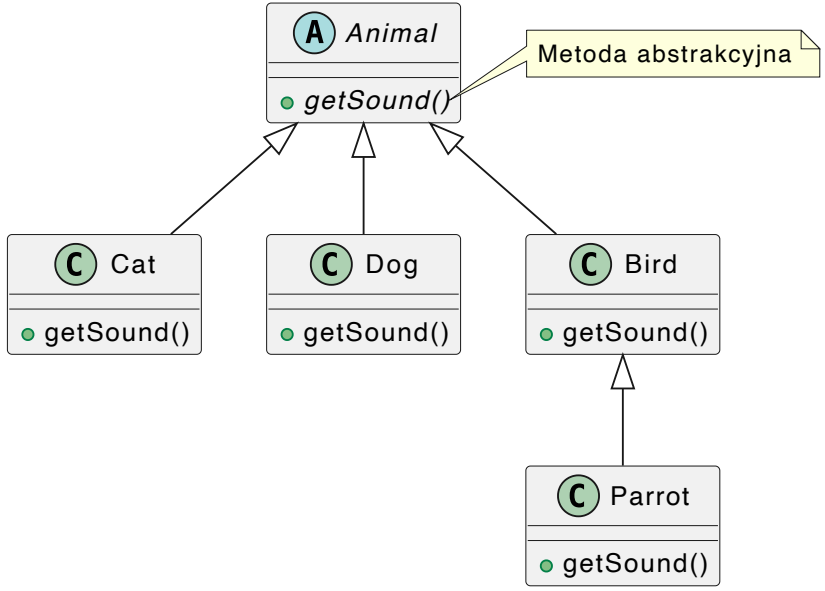
Klasy abstrakcyjne

- Niekiedy klasa bazowa istniej tylko po to, żeby dziedziczyć z niej i tworzyć referencje (do rzutowania w górę), natomiast nigdy nie stworzymy obiektu tej klasy (np. nieokreślone zwierzę) – to klasa **abstrakcyjna**.
- **Klasa abstrakcyjna**
  - **Nie można utworzyć jej instancji**.
  - Służy do **wspólnego interfejsu** i częściowego kodu współdzielonego.
  - Może zawierać **metody abstrakcyjne** oraz zwykłe metody z implementacją.
  - Klasa dziedzicząca **musi zaimplementować wszystkie metody abstrakcyjne**, chyba że też jest abstrakcyjna.



Metody abstrakcyjne

- Niekiedy klasa abstrakcyjna ma metody, o których wiemy, że nigdy się nie wywołają, ponieważ będą nadpisane w klasach pochodnych – to **metody abstrakcyjne**
- **Metoda abstrakcyjna** to metoda zadeklarowana bez implementacji – tylko z sygnaturą.
  - Może istnieć **tylko w klasie abstrakcyjnej**.
  - Tworzy **kontrakt**, który wymusza implementację w klasach dziedziczących.
  - Służy do zapewnienia **spójnego interfejsu** w hierarchii klas.
- W języku UML metody abstrakcyjne oznacza się kursywą.



Klasy i metody abstrakcyjne w C++

- W C++ metoda abstrakcyjna to **funkcja czysto wirtualna**.
- Metoda abstrakcyjna musi być zadeklarowana jako:

```
virtual void nazwa() = 0;
```

- Klasa zawierająca choć jedną metodę czysto wirtualną staje się **abstrakcyjna**.
- **Nie można tworzyć instancji** klasy abstrakcyjnej.
- Klasa pochodna musi **nadpisać wszystkie metody czysto wirtualne**, inaczej też będzie abstrakcyjna.

Przykład:

```
class Animal {
public:
    virtual string getSound() = 0;  // czysto wirtualna
};

class Dog : public Animal {
public:
    string getSound() override {
        return "Hau hau!";
    }
};
```

Klasy i metody abstrakcyjne w Javie

- Metoda abstrakcyjna:
  - **Nie ma ciała** (`{}`).
  - Musi być zadeklarowana z `abstract`.
- Klasa z przynajmniej jedną metodą abstrakcyjną musi być oznaczona jako `abstract`.
- **Nie można utworzyć instancji** klasy abstrakcyjnej.
- Klasa pochodna musi **nadpisać wszystkie abstrakcyjne metody**, lub sama być abstrakcyjna.

Przykład:

```
abstract class Animal {
    abstract String getSound();  // bez ciała
}

class Dog extends Animal {
    @Override
    String getSound() {
        return "Hau hau!";
    }
}
```

Klasy i metody abstrakcyjne w C#

- Metoda abstrakcyjna:
  - Deklarowana z `abstract` i **bez ciała**.
- Może być tylko w klasie oznaczonej jako `abstract`.
- Klasa abstrakcyjna **może mieć metody z implementacją**.
- Instancji klasy abstrakcyjnej nie można tworzyć.
- Klasa pochodna musi **nadpisać wszystkie abstrakcyjne metody**, chyba że też jest abstrakcyjna.
- Używa się słowa `override` do nadpisania.

Przykład:

```
abstract class Animal {
    public abstract string GetSound();
}

class Dog : Animal {
    public override string GetSound() {
        return "Hau hau!";
    }
}
```

Po co nam metoda abstrakcyjna?

Skoro **metoda abstrakcyjna nie ma implementacji**, to po co ją w ogóle deklarować? Czy nie lepiej po prostu nic nie pisać i zostawić to klasom pochodnym?

- Metoda abstrakcyjna nie mówi *jak coś zrobić*, tylko że *coś musi być zrobione* – to forma **kontraktu**, który **wymusza spójność** między klasami pochodnymi.
  - Tworzenie wspólnego interfejsu – metoda abstrakcyjna mówi: *„Każda klasa, która dziedziczy po mnie, musi mieć własną wersję tej metody.”*
  - Wymuszanie spójności w hierarchii – dzięki temu mamy **spójny zestaw zachowań** dla wszystkich klas pochodnych – nawet jeśli każdy obiekt robi to inaczej.
  - Wsparcie dla polimorfizmu – dzięki abstrakcyjnej metodzie można pisać kod, który działa na ogólnym typie, ale zachowuje się różnie dla każdego przypadku:

```
for(Animal animal : animals) {
    System.out.println(animal.getSound());
}
```

Przerwanie łańcucha funkcji wirtualnych

- W łańcuchu metod wirtualnych kolejne klasy **nadpisują (*override*)** metodę z klasy bazowej, a wywołanie jest rozstrzygane **dynamicznie** (na podstawie typu obiektu w czasie wykonania).
- Jeśli **nie chcemy**, żeby metoda mogła być dalej nadpisywana, możemy przerwać łańcuch wirtualny.
- Powoduje to zablokowanie możliwości nadpisywania metody w klasach pochodnych.
- Metoda nie jest już częścią łańcucha dynamicznego wiązania.
- Po co to robić?
  - Bezpieczeństwo projektu – chcemy, by pewna metoda **zawsze działała w określony sposób**.
  - Stabilność API – nie chcemy, by inni programiści modyfikowali kluczowe zachowanie.
  - Optymalizacja – kompilator może pominąć mechanizm dynamicznego wiązania.
- **C++**: Od C++11 można użyć słowa kluczowego `final`.

```
void makeSound() final override {}  // nie można nadpisać dalej
```

- **Java**: Metoda `final` nie może być dalej nadpisywana w żadnej klasie pochodnej.

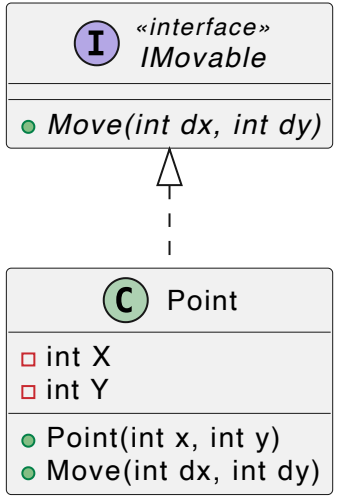
```
final void makeSound() {} // nie można nadpisać dalej
```

- **C#**: przerwanie łańcucha za pomocą `sealed` użytego przy metodzie `override`.

```
public sealed override void makeSound() {}  // nie można nadpisać dalej
```

Typ interfejsowy

- Typ interfejsowy (interfejs) jest programistyczną realizacją pojęcia „interfejs klasy” wprowadzonego przy okazji hermetyzacji.
- **Typ interfejsowy** to **abstrakcyjny typ**, który określa **zbiór metod (kontrakt)**, jakie obiekt musi posiadać, **nie definiując ich implementacji**.
- **Typ interfejsowy** to **nazwa interfejsu**, traktowana jak typ, który można wykorzystać:
  - do deklaracji zmiennych,
  - parametrów metod,
  - zwracanych wartości,
  - kolekcji,
  - itp.



Typ interfejsowy w Javie

- Słowa kluczowe `interface` i `implements`

```
interface Movable {
    void move(int dx, int dy);
}

class Point implements Movable {
    // ...
    public void move(int dx, int dy) {
        this.x += dx;
        this.y += dy;
    }
}
```

- Interfejs nie zawiera implementacji (choć od Java 8 może zawierać implementację domyślną metod).
  - Może też zawierać statyczne, finalne pola.
- Definiuje **co obiekt potrafi robić**, ale nie **jak** to robi.
- Metody interfejsu są domyślnie publiczne abstrakcyjne.
- Klasa **musi zaimplementować wszystkie abstrakcyjne metody** z interfejsu.
- Może implementować **wiele interfejsów**.
- Interfejsy mogą **dziedziczyć po innych interfejsach**.

Typ interfejsowy w C#

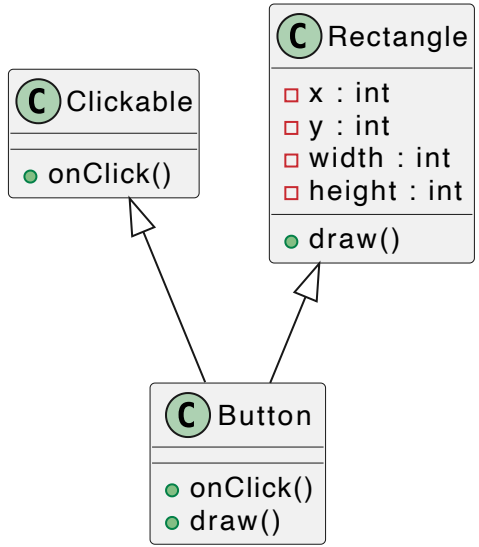
- Analogicznie jak w Javie:

```
interface IMovable {
    void Move(int dx, int dy);
}

class Point : IMovable {
    // ...
    public void Move(int dx, int dy) {
        this.x += dx;
        this.y += dy;
    }
}
```

- Podobnie jak w Javie:
  - interfejs definiuje **co obiekt może robić**,
  - ale nie musi definiować **jak to robi**.
- Metody domyślnie są publiczne i bez implementacji.
- Od C# 8.0 można definiować metody z domyślną implementacją oraz pola statyczne.
- Implementacja interfejsu przy pomocy `:`.

Wielodziedziczenie

- **Wielodziedziczenie** (ang. *multiple inheritance*) to mechanizm, w którym **jedna klasa dziedziczy po więcej niż jednej klasie bazowej**.
- Klasa `Button` dziedziczy **jednocześnie** po klasie `Clickable` i `Rectangle`.
- Wielodziedziczenie pozwala tworzyć **bardziej elastyczne hierarchie** i umożliwia **ponowne użycie kodu z różnych klas**.
- W praktyce jednak – może prowadzić do problemów i konfliktów.



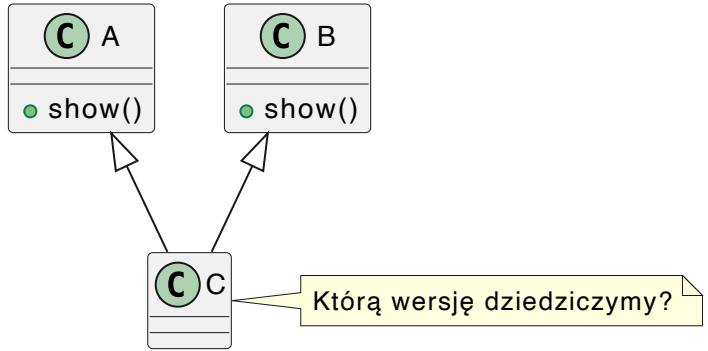
Składnia wielodziedziczenie w C++

```
class Rectangle {
public:
    void draw() { cout << "Rectangle::draw" << endl; }
};

class Clickable {
public:
    void onClick() { cout << "Clickable::onClick" << endl; }
};

class Button : public Rectangle, public Clickable {};

int main() {
    Button btn;
    btn.draw();
    btn.onClick();
}
```

Problemy związane z wielodziedziczeniem

- Konieczność rozstrzygania konfliktów w razie więcej niż jednej implementacji:

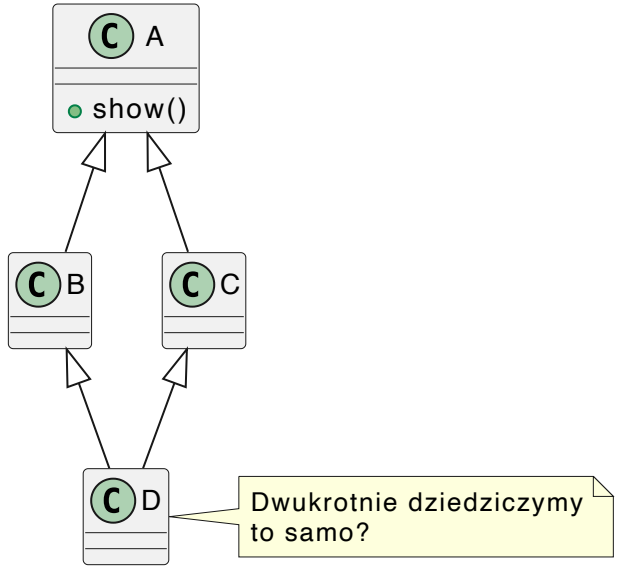
```
class A {
public:
    void show () { cout << "A"; }
};

class B {
public:
    void show () { cout << "B"; }
};

class C : public A, public B {
};

// Która wersja metody zostanie wywołana?
C c;
c.show(); // ❌ Błąd!

// Należy jawnie określić pochodzenie

obj.A::show(); // ✅ ok
obj.B::show(); // ✅ ok
```



Problemy związane z wielodziedziczeniem

- Problem diamentu (*diamond problem*) – w razie wielokrotnego dziedzczienia po tej samej klasie (pośrednio).

```
class A {
public:
    void show () { cout << "A"; }
};

class B : public A {};

class C : public A {};

class D: public B, public C {};
```

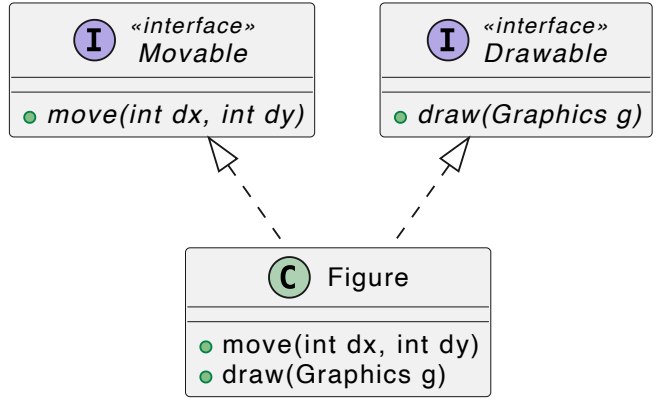


- Dwukrotnie dziedziczymy po `A` (przez `B` i przez `C`), zatem dwukrotnie odziedziczyliśmy metodę `show`.
- Rozwiązanie – wirtualne dziedziczenie: `class B : virtual public A {};`

Wielodziedziczenie interfejsów

- **Java** i **C#** nie wspierają wielodziedziczenia klas, ale pozwalają na implementowanie wielu interfejsów.

```
interface Movable {
    void move(int dx, int dy);
}
interface Drawable {
    void draw(Graphics g);
}
class Figure implements Movable, Drawable {
    public void move(int dx, int dy) {
        // ...
    }
    public void draw(Graphics g) {
        // ...
    }
}
```

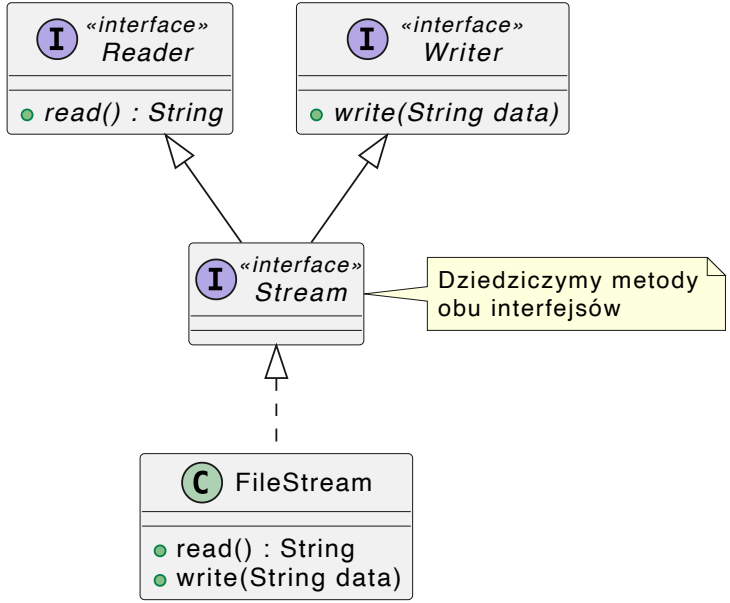


- Implementując dwa interfejsy, deklarujemy implementację wszystkich ich metod.

Wielodziedziczenie interfejsów

- **Java** i **C#** pozwalają na wielodziedziczenie interfejsów.

```
interface Reader {
    String read();
}
interface Writer {
    void write(String data);
}
interface Stream extends Reader, Writer {
    // może również dodawać własne metody
}
class FileStream implements Stream {
    public String read() {
        // ...
    }
    public void write(String data) {
        // ...
    }
}
```



- Interfejs `Stream` jest sumą interfejsów `Reader` i `Writer`.